

Introduction

Real-time rendering systems rely on consistent frame delivery to maintain visual smoothness and responsiveness. Typically, frames per second (FPS) is the most common metric that is used as a performance metric, and while it does well at providing a general idea of how "fast" a game is running or when comparing raw performance capacity, it fails to capture short-term inconsistencies in frame delivery. These inconsistencies are better represented by frame time variance, which more directly reflects perceived stuttering even when average FPS remains high.

Modern operating systems such as Linux use general-purpose scheduling algorithms designed primarily to maximize overall CPU utilization and fairness among processes. In particular, Linux employs the Completely Fair Scheduler (CFS), which distributes CPU time proportionally across runnable tasks. While this approach improves system-wide fairness, it does not inherently prioritize low-latency execution for individual workloads. As a result, latency-sensitive applications such as games and real-time rendering systems may experience scheduling delays under CPU contention, leading to irregular frame pacing.

This project investigates how CPU scheduling behavior in Linux affects frame time stability in a controlled OpenGL rendering workload. The focus is on measuring how different scheduling configurations influence latency-sensitive rendering performance under varying levels of CPU contention.

Overview and Methods

In order to analyze the effects of CPU scheduling in Linux, I plan to use a machine dedicated to running Ubuntu Latest, and using control and experimental groups of scheduling configurations with the default being how it would run on a practically fresh Linux installation,

and the others including having the renderer be prioritized, the renderer on an isolated core, a mix, and likely other configurations too. I want to show that CPU scheduling decisions significantly affect frame time stability in real-time workloads under CPU stress, and that priority and CPU affinity optimizations will reduce frame time variance compared to default scheduling behavior. So, the project will consist of the following code:

1. **A custom OpenGL rendering window** developed in C++ using OpenGL and GLFW (Yay I get to learn C++ a semester early!). It'll display a fixed scene with a controlled number of draw calls and shader operations to ensure a consistent amount of work across all runs. Frame timing will be measured using system timers, allowing precise capture of frame durations. These will be logged to a CSV file for analysis and reflection on the results.
2. **A configurable CPU stressor** will be implemented to simulate real-world system load. To do this, I plan to have it spawn multiple threads that will work on a configurable computational load, similarly to the work we did with threads previously in class. The number of threads should be adjustable so that I can analyze varying levels of stress.

Bibliography

- Wong, C. S., Tan, I., Kumari, R. D., & Lam, J. W. (2008). Towards achieving fairness in the Linux scheduler. *Operating Systems Review*, 42(5), 34–43.
- Yoon, K., Jeong, E., Kang, W., et al. (2025). Worst case response time analysis for completely fair scheduling in Linux systems. *Real-Time Systems*, 61, 118–158.
- Wong, C. S., Tan, I., Kumari, R. D., Lam, J. W., & Fun, W. (2008). Fairness and interactive performance of O(1) and CFS Linux kernel schedulers. In *Proceedings of the International Symposium on Information Technology (ITSim)*, 3, 1–8.